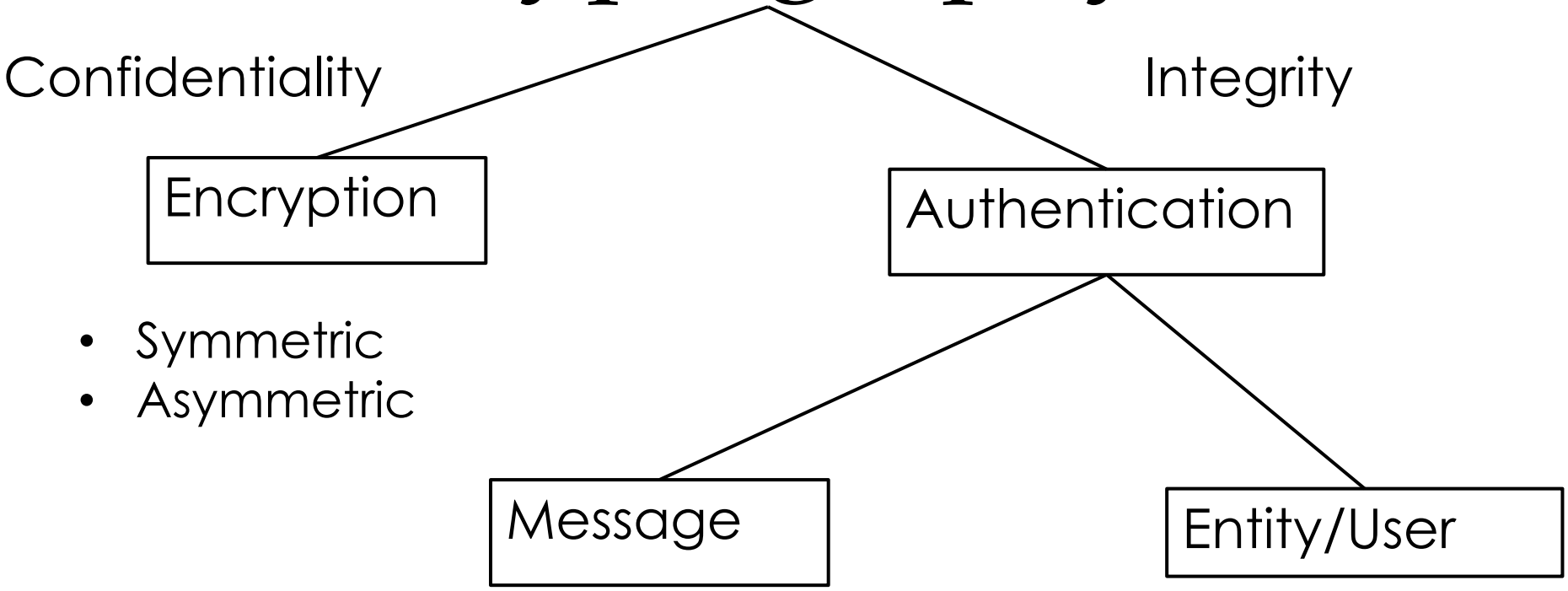
بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِيْمِ

اَلْحَمْدُ لِلّٰهِ رَبِّ الْعَالَمِيْنَ

وَالصَّلٰوةُ وَالسَّلَامُ عَلٰى خَاتَمِ النَّبِيِّيْنَ

Message Authentication

Cryptography



- Symmetric
- Asymmetric

- Authentication Codes
- Digital SIgnatures

- Challenge Response
- Zero Knowledge

Message Integrity (Authentication)

Protects against active attacks	
Verifies received message is authentic	• Contents have not been altered • From authentic source • Timely and in correct sequence
Can use conventional encryption	• Only sender and receiver share a key

Encryption cannot **always** provide integrity/authentication (protection against active/modification attack **e.g. by data blocks reordering)**

Message Authentication Without Confidentiality

Message encryption by itself does not provide authentication

Message authentication is provided as a separate function from message encryption

Situations in which message authentication without confidentiality may be preferable include:

1. **Applications in which the same message is broadcast to a number of destinations (e.g. transactions in cryptocurrency)**

2. **One side has a heavy load and cannot afford the time to decrypt all incoming messages**

3. **Authentication of a computer program in plaintext**

there is a place for both authentication and encryption

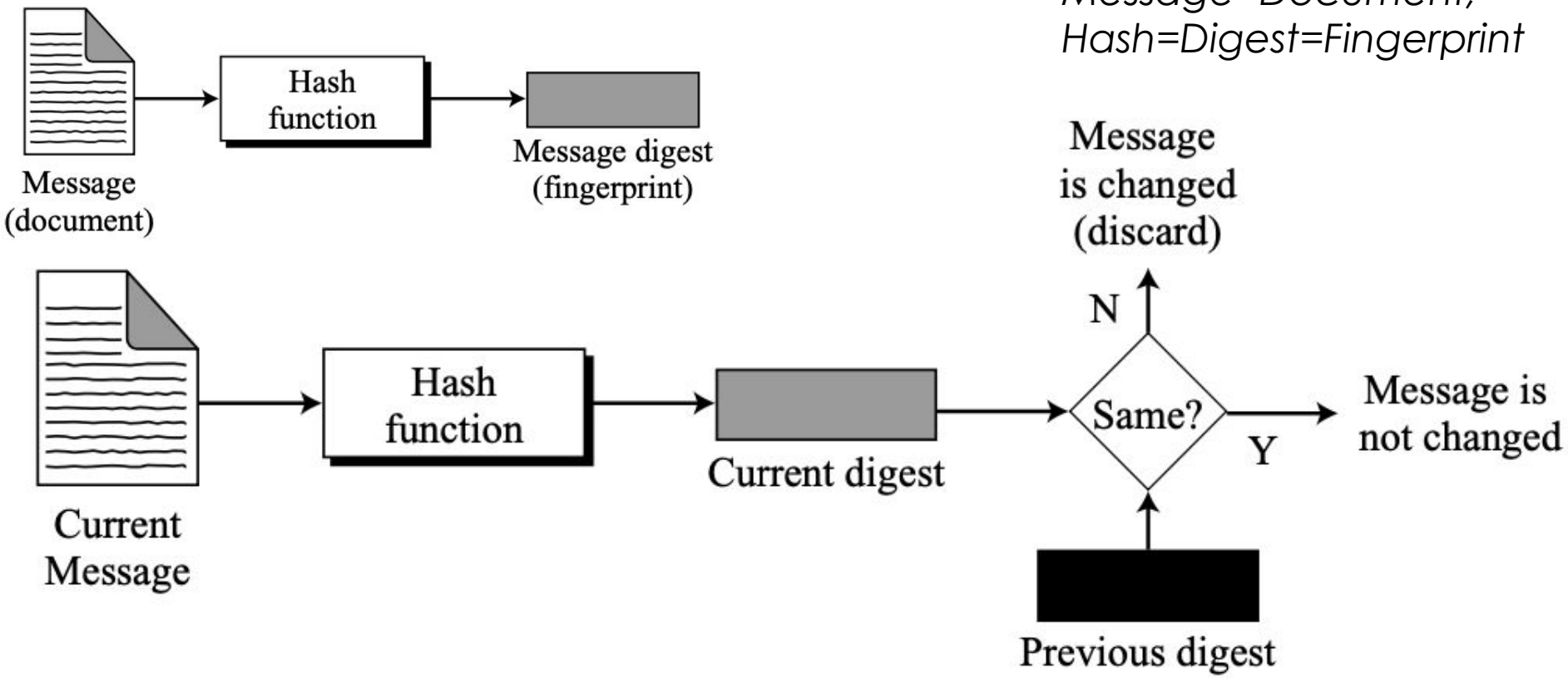
Hashing for Authentication

Hash value can be used for Authentication **(if kept safe) e.g**

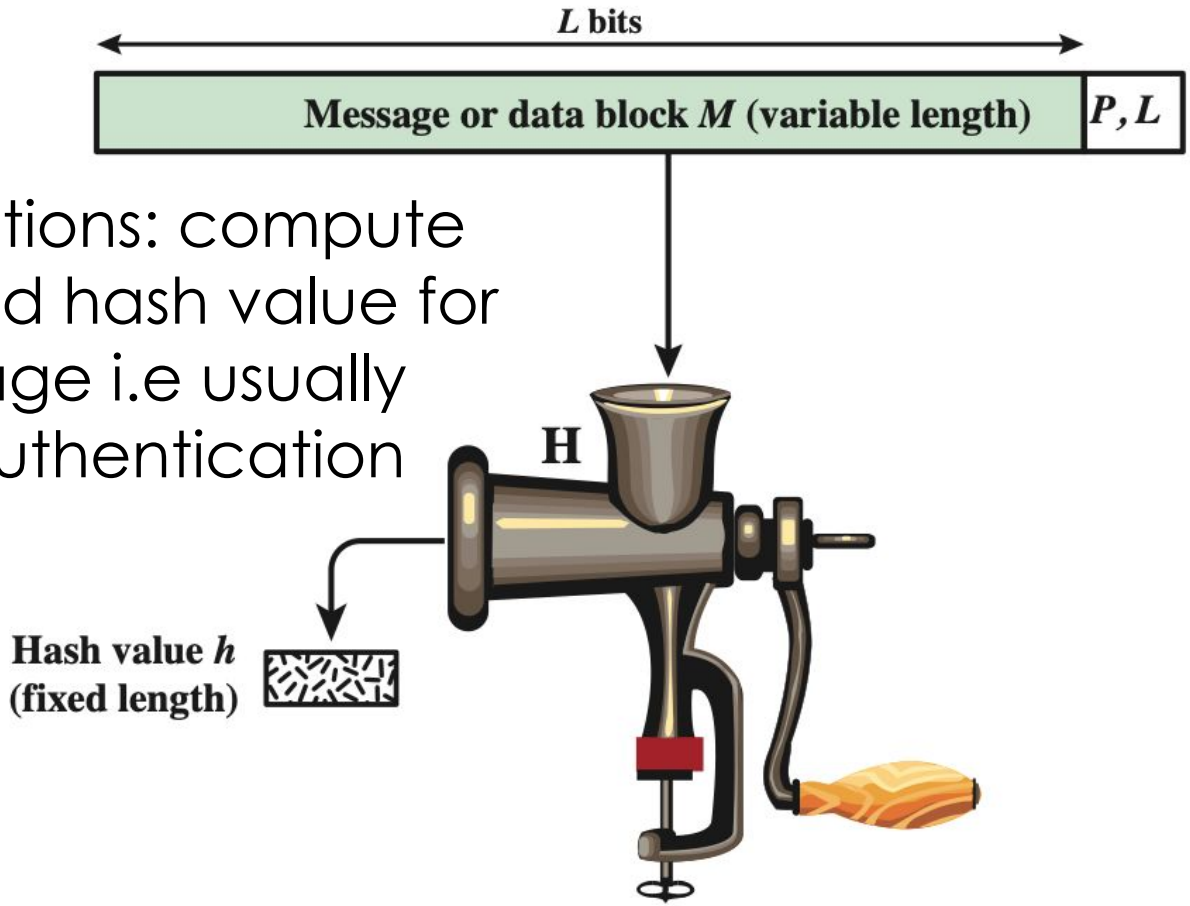
Hash stored (for verification later)

user passwords hashed on server (keeping them secret)

Message=Document,
Hash=Digest=Fingerprint



Hash functions: compute the fix sized hash value for the message i.e usually used as authentication code



P, L = padding plus length field

Figure 2.4 Cryptographic Hash Function; $h = H(M)$

Simple Hashing

Hash value: Flx sized value/digest depending on the message/data

Simple hash value examples

- modulo operator over complete message M:
  M mod 2^n → for n bit hash value
- XOR n bit blocks sequentially for n bit hash (might need padding for one block)
- modulo 2^n addition for n bit blocks

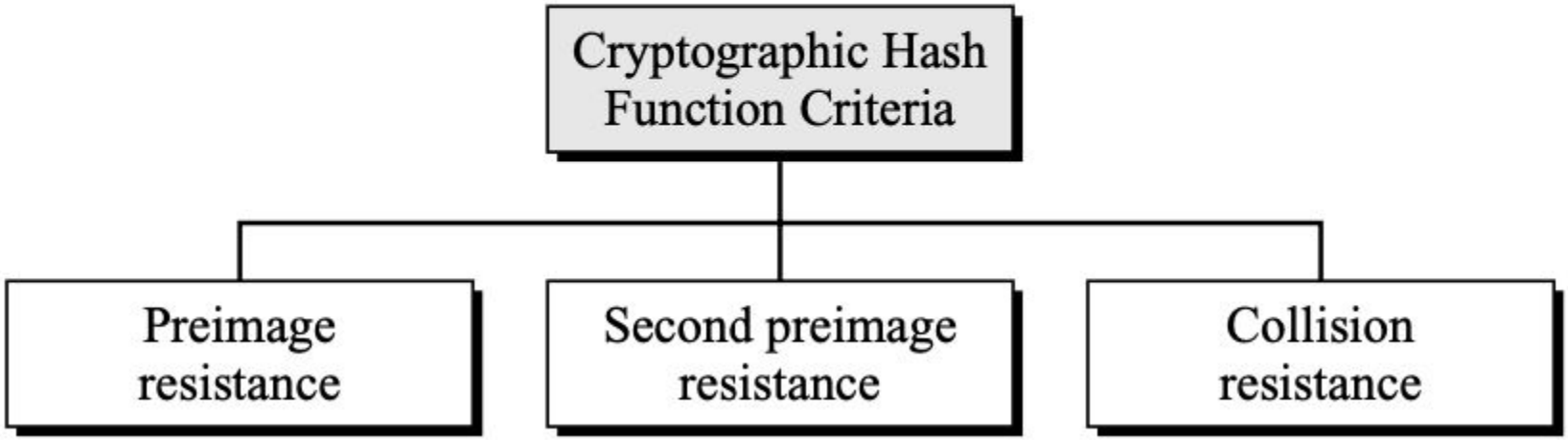
	Bit 1	Bit 2	• • •	Bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
	•	•	•	•
	•	•	•	•
	•	•	•	•
Block m	b_{1m}	b_{2m}		b_{nm}
Hash code	C_1	C_2		C_n

Figure 21.1 Simple Hash Function Using Bitwise XOR

Hash Features

To be used for authentication a hash function must be a secure (cryptographic hash function)

Criteria of a cryptographic hash function



Secure Hashing

Features:

1. A message must always give the same hash value
2. Original message must not be retrievable (hard to retrieve) using the hash value (**pre-image resistance**)
3. Having the original message it's hard to find another message with same hash (2nd **pre-image resistance**)
4. No two messages (found easily) must give same hash value (**collision resistance**)

To be useful, a hash function H must have the following properties:

- Can be applied to a block of data of any size
- Produces a fixed-length output
- H(x) is relatively easy to compute for any given x
- One-way or pre-image resistant
  Computationally infeasible to find x such that H(x) = h
- Computationally infeasible to find y ≠ x such that H(y) = H(x)
- Collision resistant or strong collision resistance
  Computationally infeasible to find any pair (x,y) such that H(x) = H(y)

Brute force attacks

for n bits in hash

Brute force for both preimage attacks needs to try changing messages and checking that has complexity proportional to trying (2^n) permutations possible for the has value

For collision attack since a combination of any two samples needs to collide, complexity gets proportional to 2^(n/2) i.e. lower. (birthday paradox: 23 required for birthday collision)

Attack	*Order*
Preimage	2^n
Second preimage	2^n
Collision	$2^{n/2}$

Hashing Functions

Most Secure Hashing Algorithms use:

Padding + Block segmentations + Circular shifts + inter block operations e.g. bitwise XOR or Mod addition

Random chance probability to get a right hash value 1/2^n for n bits is reduced due to structure of data e.g. higher bits being 0 for ASCII.

An issue handled by circular shifts

Common Hash Algorithms

MD5

- 128-bit digest
- Fast but broken: collision attacks are practical
- Vulnerable mainly due to short digest length

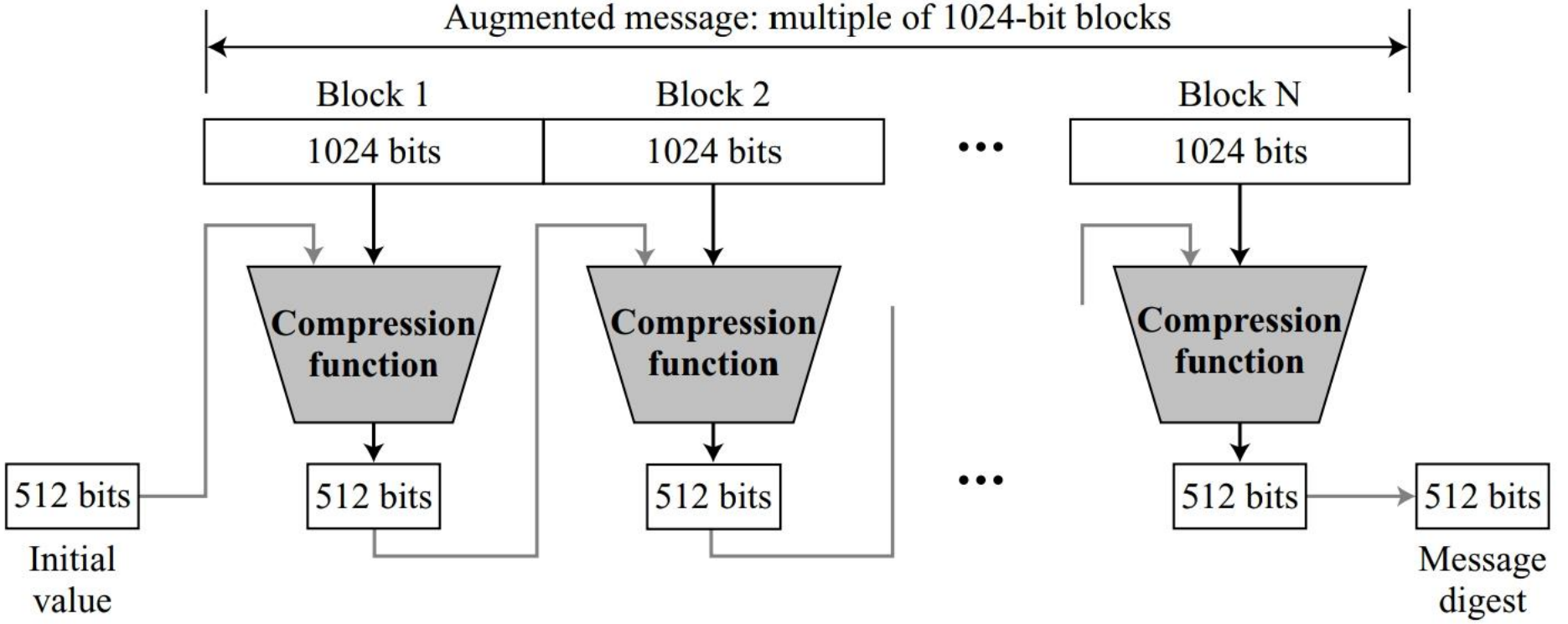
SHA-2 Family

- Includes SHA-256, SHA-384, SHA-512
- Digest lengths: 256/384/512 bits
- Currently secure and widely used (TLS, PKI)
- No practical collision attacks known

SHA 512

1024 bit blocks compressed to 512 bits and processed sequentially to get the 512 bit digest for the complete data

Figure 12.6 *Message digest creation SHA-512*



Digest initialization

Initialized as 8, 64 bit words = 8*64=512 (named A to H)

Table 12.2 *Values of constants in message digest initialization of SHA-512*

Buffer	*Value (in hexadecimal)*	*Buffer*	*Value (in hexadecimal)*
A_0	6A09E667F3BCC908	E_0	510E527FADE682D1
B_0	BB67AE8584CAA73B	F_0	9B05688C2B3E6C1F
C_0	3C6EF372EF94F82B	G_0	1F83D9ABFB41BD6B
D_0	A54FE53A5F1D36F1	H_0	5BE0CD19137E2179

The values are calculated from the **first eight prime** numbers (2, 3, 5, 7, 11, 13, 17, and 19).

Each value is the fraction part of the square root of the corresponding prime number after converting to binary and keeping only the first 64 bits

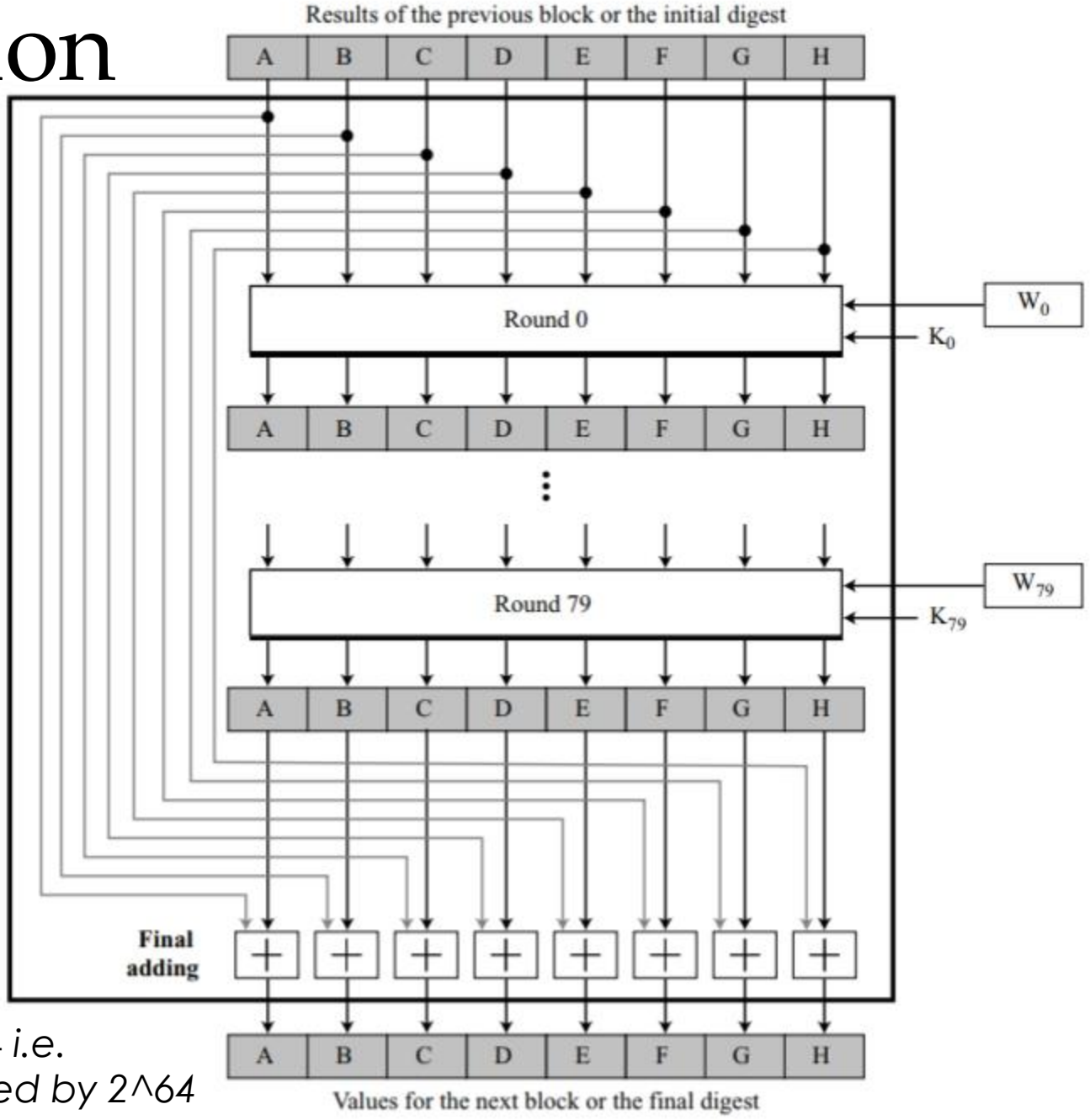
Compression

Initialized digest is passed through 80 rounds each involving a word from the data block.

The **data block comprises 16 words that are expanded to 80** for the compression function

Final addition modulo 2^64 *i.e. remainder when sum divided by 2^64*

Figure 12.10 *Compression function in SHA-512*

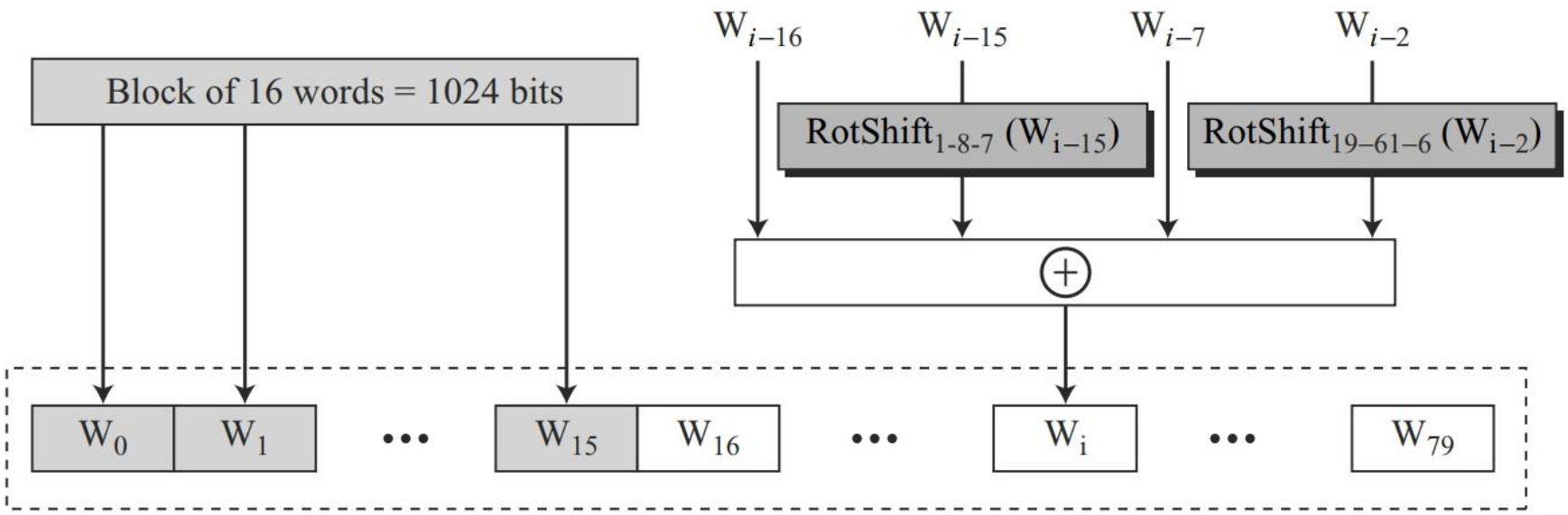


Block expansion

before compression:

each 1024 bit data block is segmented in 16, 64 bit words (16*64=1024) and then expanded to 80 words

Figure 12.9 *Word expansion in SHA-512*



$RotShift_{1\text{-}m\text{-}n}(x)$: $RotR_l(x) \oplus RotR_m(x) \oplus ShL_n(x)$

$RotR_i(x)$: Right-rotation of the argument x by i bits
$ShL_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

Rounds

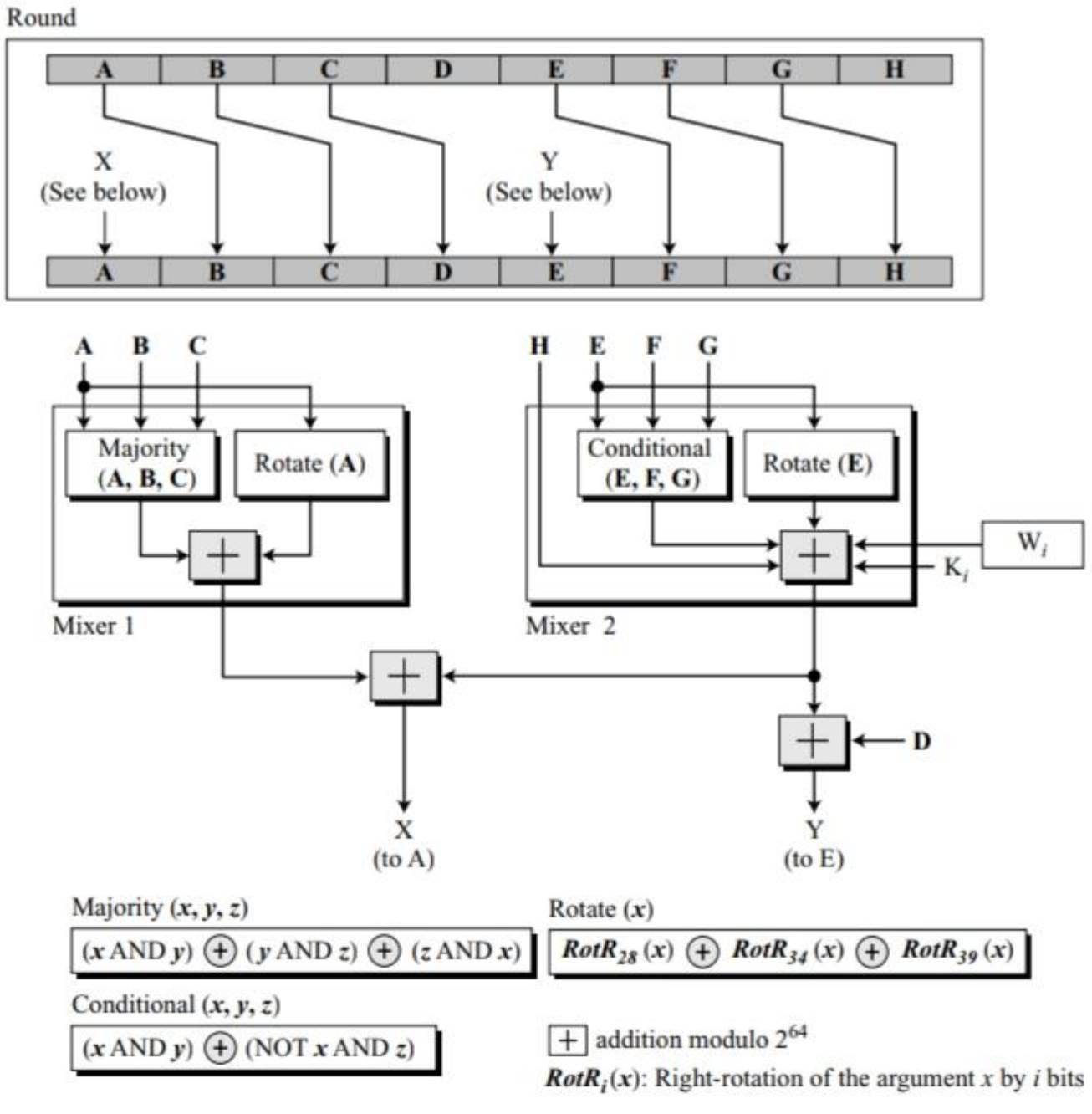
Within each round the digest words from previous block (or initialized) are right shifted by one word except for the new words A and E

K1 to K80, predefined word constants (can be keys)

A Word from the data block is used (by addition modulo 2^64) to create A and E

i.e. remainder when the sum is divided by 2^64

Figure 12.11 *Structure of each round in SHA-512*



Comparison of SHA Parameters

	SHA-1	SHA-224	SHA-256	SHA-384	SHA-512	SHA-512/224	SHA-512/256
Message size	$< 2^{64}$	$< 2^{64}$	$< 2^{64}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$	$< 2^{128}$
Word size	32	32	32	64	64	64	64
Block size	512	512	512	1024	1024	1024	1024
Message digest size	160	224	256	384	512	224	256
Number of steps	80	64	64	80	80	80	80
Security	80	112	128	192	256	112	128

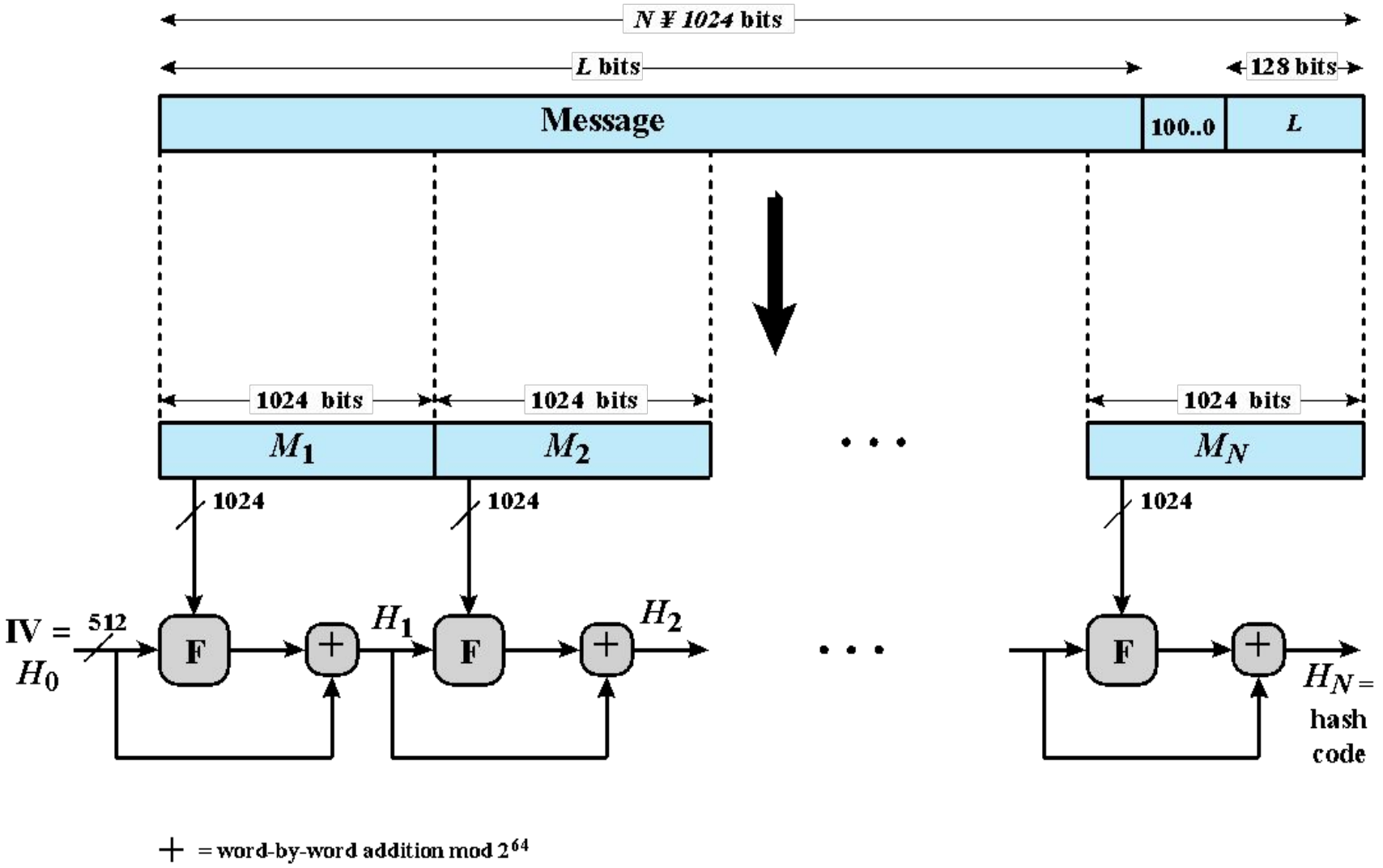


Figure 21.2 Message Digest Generation Using SHA-512

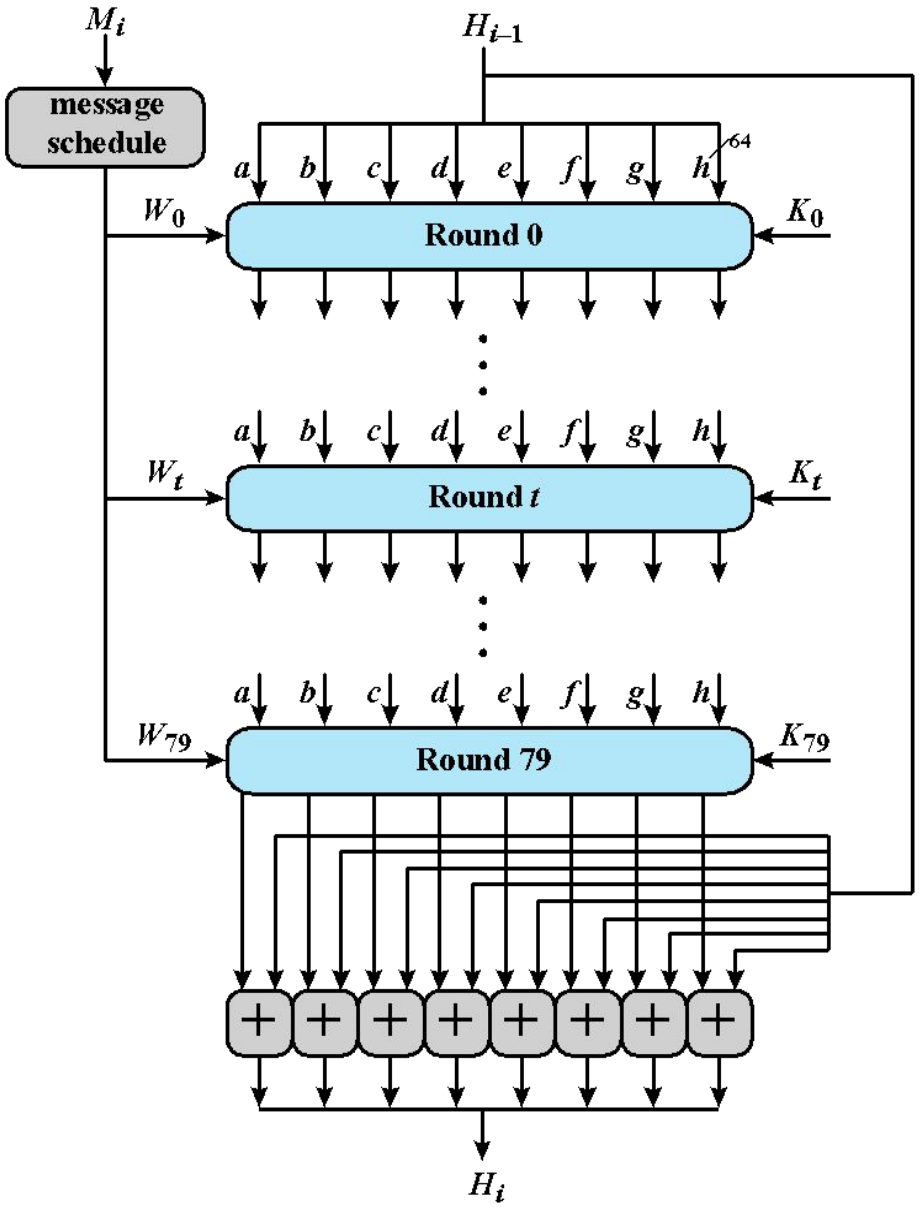


Figure 21.3 SHA-512 Processing of a Single 1024-Bit Block

Secure Hash Algorithm (SHA)

- SHA was originally developed by NIST
- Published as FIPS 180 in 1993
- Was revised in 1995 as SHA-1
  - Produces 160-bit hash values
- NIST issued revised FIPS 180-2 in 2002
  - Adds 3 additional versions of SHA
  - SHA-256, SHA-384, SHA-512
  - With 256/384/512-bit hash values
  - Same basic structure as SHA-1 but greater security
- The most recent version is FIPS 180-4 which added two variants of SHA-512 with 224-bit and 256-bit hash sizes

SHA-3

- SHA-2 shares same structure and mathematical operations as its predecessors and causes concern
- Due to time required to replace SHA-2 should it become vulnerable, NIST announced in 2007 a competition to produce SHA-3

Requirements:

- Must support hash value lengths of 224, 256,384, and 512 bits
- Algorithm must process small blocks at a time instead of requiring the entire message to be buffered in memory before processing it

Resistant to:

- Preimage
- Collision

Secure Hashing

- Provides integrity if hash kept secure
- Hides passwords for storing

Hashing with Salt

Provides integrity even if shared across insecure channel

Hashing with secret key

Avoids dictionary attack on preimage

Digital Signature

Hash encrypted using private (asymmetric) key

Authentication Codes

Hash encrypted using pre-shared (symmetric) key

Salting

Dictionary attack on preimage:
precomputed hash values for a list of known messages (as rainbow tables) can be used to lookup the hash value and find the corresponding message.

Common attack to find passwords from hash values

To avoid this, a unique random value (salt) is added to each message (password) before hashing.

Same salt value used for hashing initially as well as for verification

As a result, rainbow tables become useless, and attackers must brute-force each password individually.

Even for known salt values the rainbow attack is useless as it needs precomputation of hash values

Hashing without secret key

Secure hashing without incorporating a secret key can provide integrity only:

- if shared out of band
- or a over secure channel

e.g.

- saved on server for protecting change over network
- saved in centralized (protected) IDS to verify files over vulnerable network

If hash value is shared in band it can be changed and replaced according to the modified content

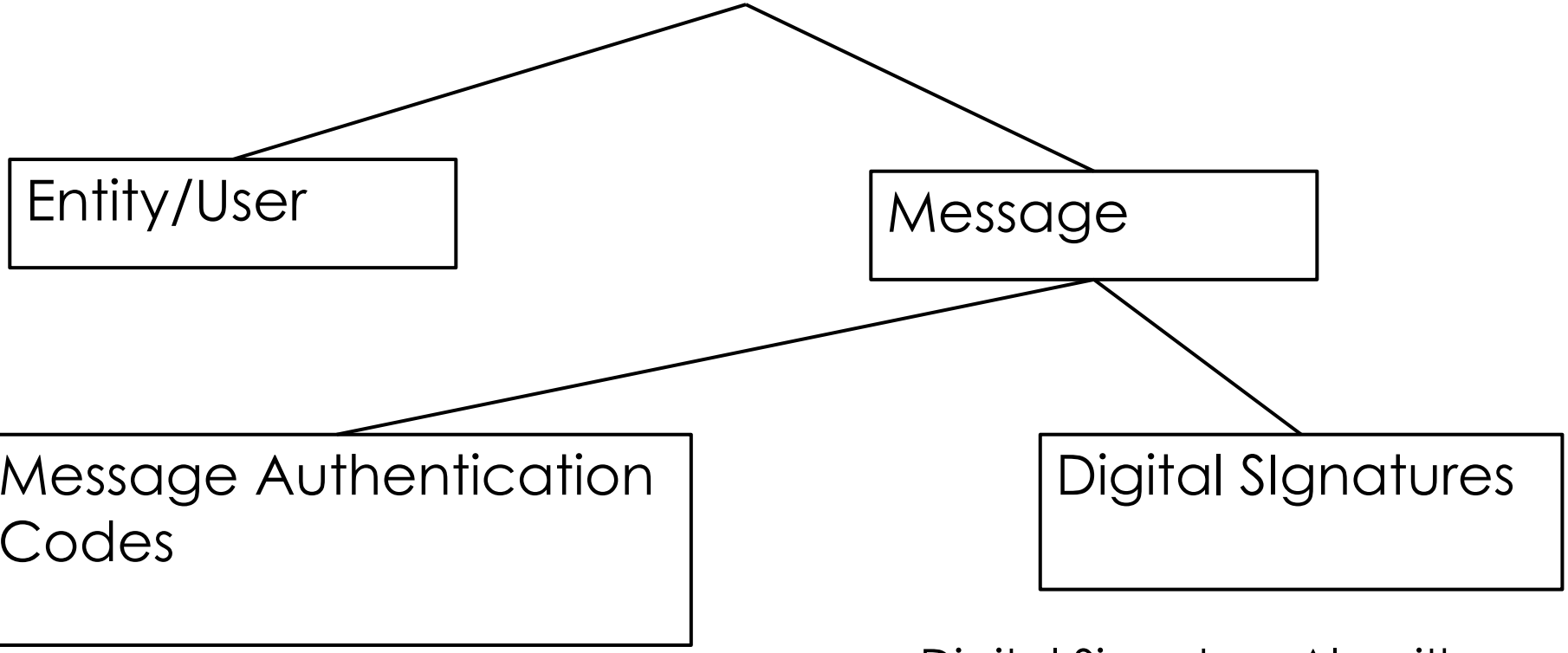
Hashing with Secret

Incorporates a secret key while hashing

so that attacker cannot update the hash value on modifying the message

Hence the hashed value using key can be transmitted in band besides the message over unsafe channel

Authentication



- Hash Function incorporating shared key
- Encrypt Hash value with shared key
- Post/pre append shared key before hashing

- Digital Signature Algorithm
- RSA Digital Signature Algorithm
- Elliptic Curve Digital Signature Algorithm

Message authentication code

Incorporates a shared (symmetric) secret key before hashing, in order to provide integrity and authentication

approaches for MAC:

1) HASH computed by a function applied over both, the content and the key
2) Encrypt the HASH value with a secret key (slow)
3) Pre/post append the key to the content before hashing (efficient) e.g. HMAC (HASHED MAC)

Digital signature uses asymmetric encryption for encrypting the hash (more later)

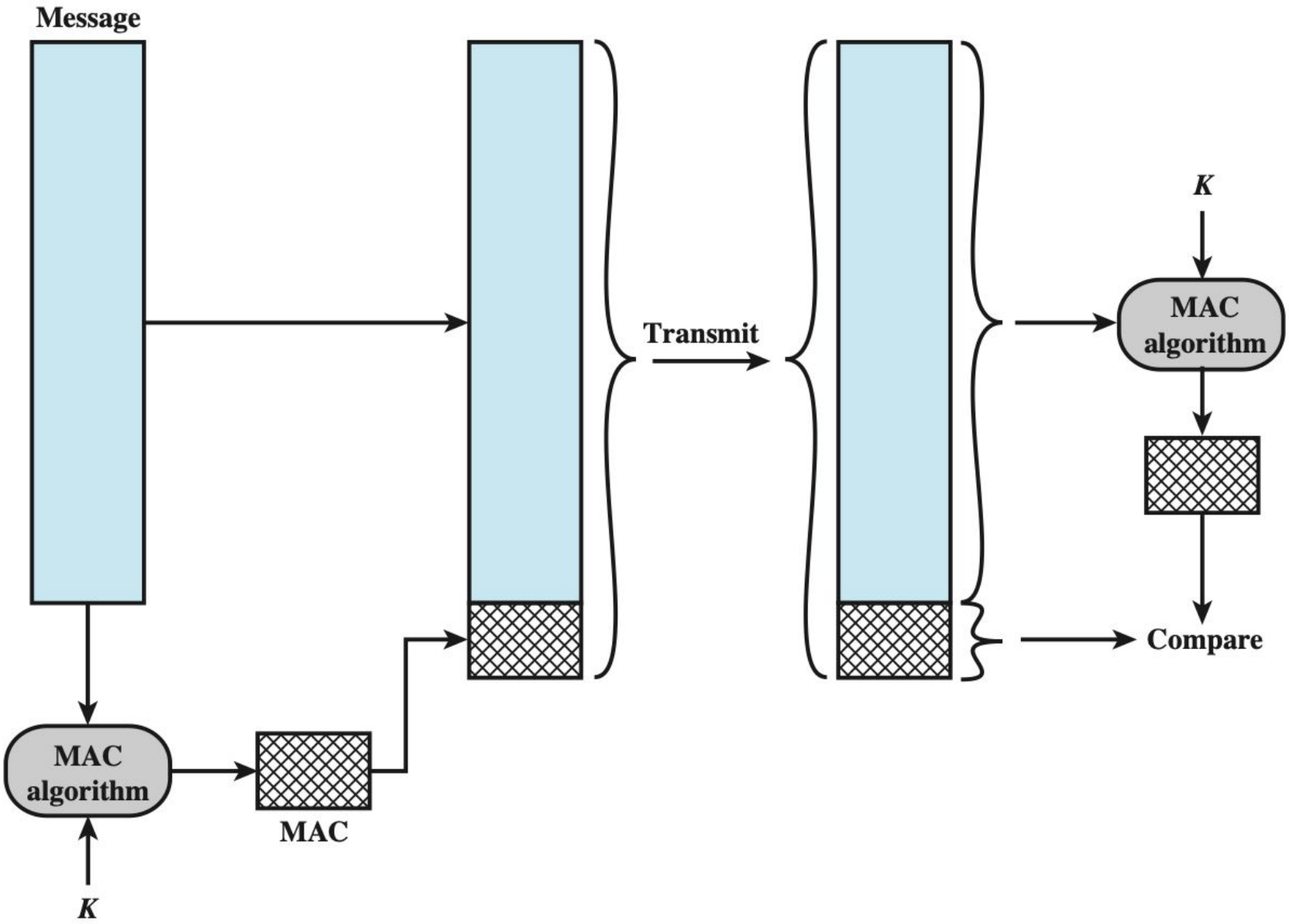
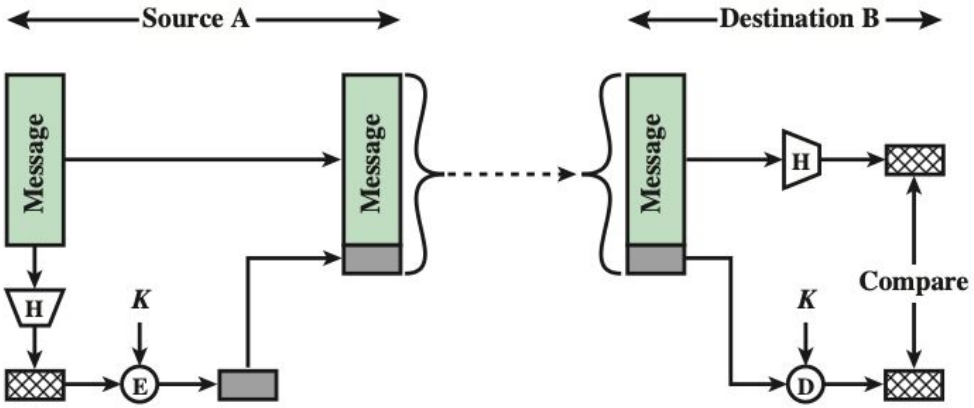
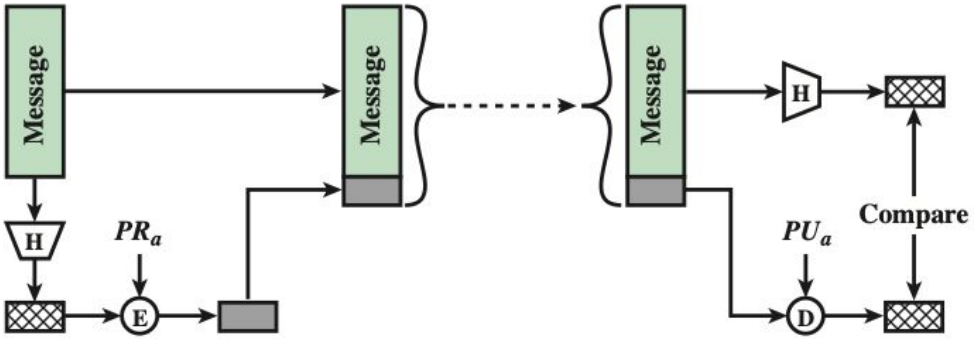


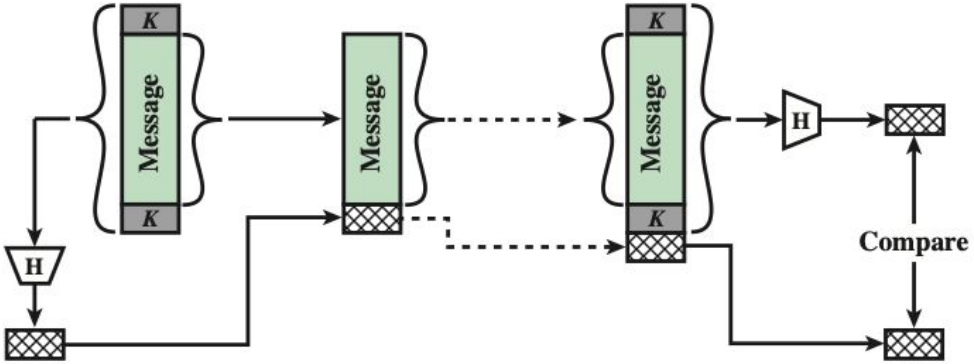
Figure 2.3 Message Authentication Using a Message Authentication Code (MAC).



(a) Using symmetric encryption



(b) Using public-key encryption



(c) Using secret value

Figure 2.5 Message Authentication Using a One-Way Hash Function.

Digital Signatures

- NIST FIPS PUB 186-4 defines a digital signature as:

  "The result of a cryptographic transformation of data that, when properly implemented, provides a mechanism for verifying origin authentication, data integrity and signatory non-repudiation."

- Thus, a digital signature is a data-dependent bit pattern, generated by an agent as a function of a file, message, or other form of data block

- FIPS 186-4 specifies the use of one of three digital signature algorithms:
  - Digital Signature Algorithm (DSA)
  - RSA Digital Signature Algorithm
  - Elliptic Curve Digital Signature Algorithm (ECDSA)

Digital Signature
Sender generates a hash value for the message with a secure hash function.

That hash value, with sender's private key, are input to a digital signature generation algorithm that produces a short data block as digital signature.

The message is sent with the signature.

Receiver calculates a hash value for the message and inputs the hash value, the sender's public key and signature to a verification algorithm.

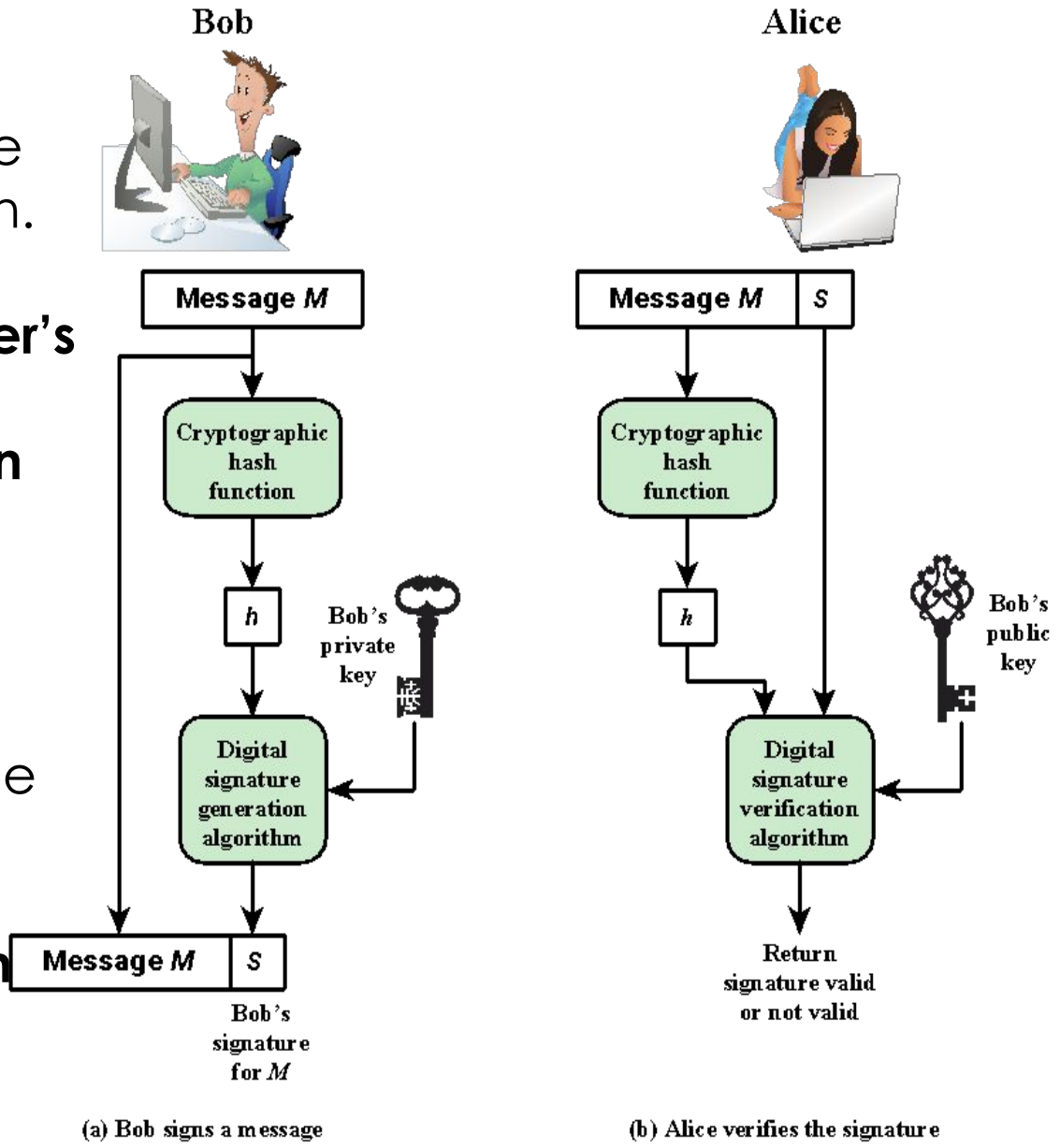


Figure 2.7 Simplified Depiction of Essential Elements of Digital Signature Process

Public Key Infrastructure

User (e.g an HTTPS server) prepares an **unsigned certificate** that includes the **user ID and user's public key**

User provides the unsigned certificate to a **Certification Authority (CA)** securely. (e.g. using a **registered e-mail**)

CA creates a d**igital signature (for the unsigned certificate)** and attaches the signature to the unsigned certificate and returns a a signed certificate to the user

The user provides the signed certificate to any other receiver who may verify that the certificate is valid using the hash of certificate (without signature) and **CA's public key** with a **signature verification** algorithm.

CAs are **heirarchial**: public keys of leaf CAs signed by root CAs which eventually are embedded in browsers/OS

X.509 universally accepted standard for formatting public-key certificates. used by IPsec, TLS, SSH

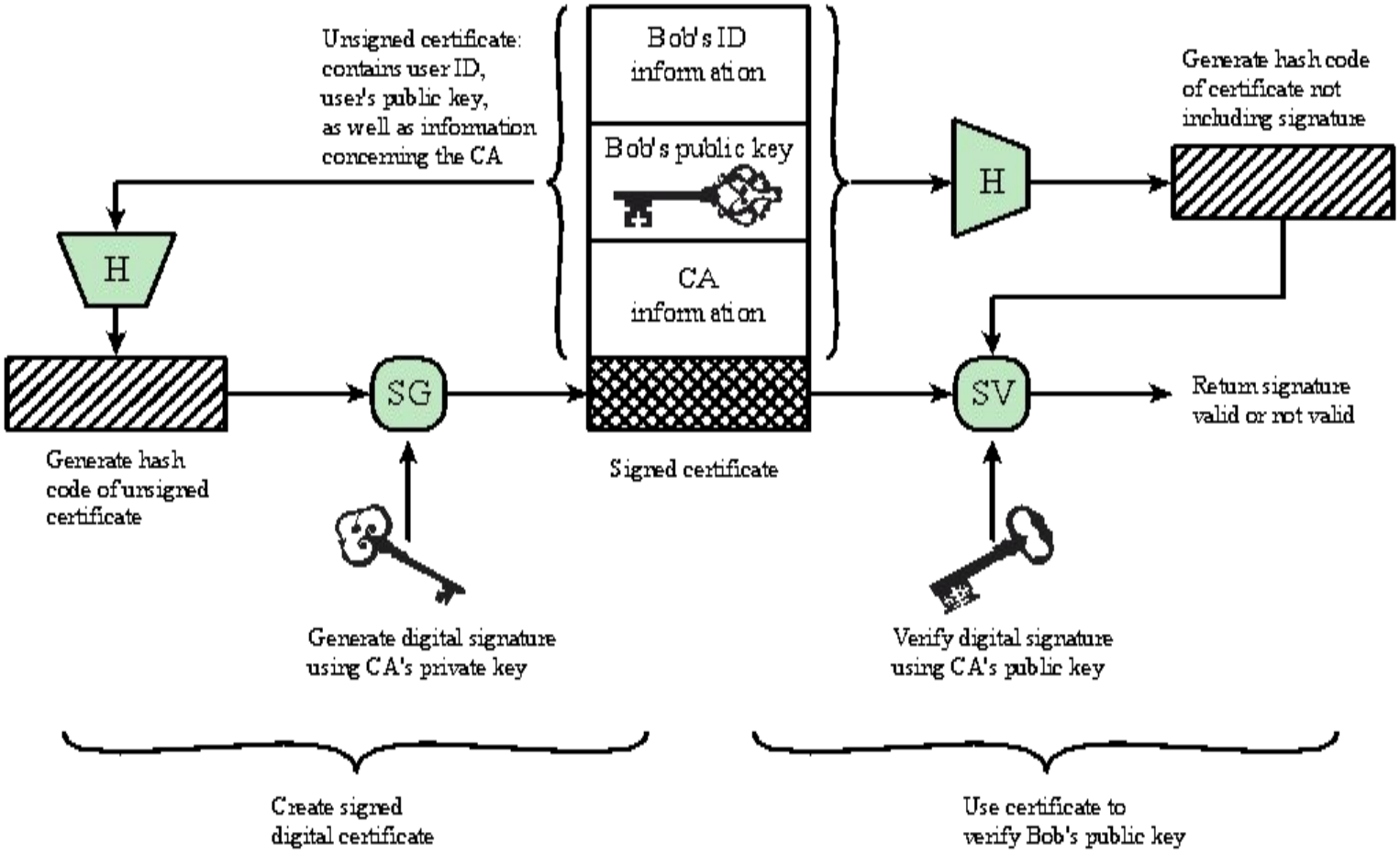


Figure 2.8 Public-Key Certificate Use

Certificates Exercise

For following Identify the key elements in the certificates, including the

1. **owner's name**
2. **public key,**
3. **validity dates,**
4. **the name of the CA that signed it,**
5. **the type and value of signature.**

1. Using your Web browser, visit any secure Web site (i.e., one whose URL starts with "https"). Examine the details of the X.509 certificate used by that site. This is usually accessible by selecting the padlock symbol (besides address bar)

2. View list of root CAs with their public keys embedded in your browser (e.g. in Chrome root store). Examine details of each

chrome://certificate-manager/crscerts

Block Chain

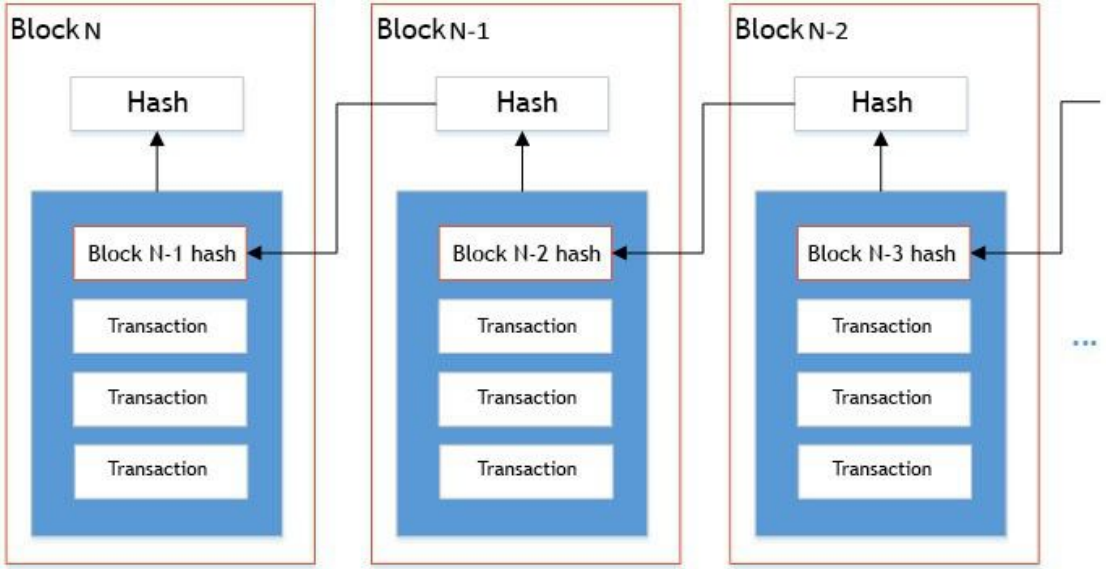
Ledger: a record of transaction details in order, e.g used for banking and tracking assets.

Block chain: chain of blocks containing transaction details.

- Every block contains transaction data with its hash or signature.
- The block is broadcast to all nodes in the distributed, peer-to-peer network
- Each new block includes hash of the previous block (before computing its own hash) linking blocks together into a chain.

Because each block depends on the previous block's hash, changing any record breaks all subsequent hashes.

To alter a block, an attacker would need to recompute the entire chain on most network nodes, which is computationally infeasible.



Hashcat

Widely used tool for cracking passwords

Includes brute force and dictionary attacks on hashing

Included on Kali linux

Available for windows and linux